FORMAT FOR PREPARATION OF PROJECT REPORT

FOR

WEB PROGRAMMING

LUMIERE HOSPITAL AND CLINIC MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

2116240701409 - RAGHUL M

Domain: Web Development / Healthcare Informatics

Academic Year: 2026-27

October, 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "LUMIERE HOSPITAL AND CLINIC MANAGEMENT SYSTEM" is the bonafide work of Raghul M (2116240701409), who carried out the Web Programming project work under the prescribed academic guidance.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	SOFTWARE AND HARDWARE REQUIREMENTS	5
2	FULL VIEW OF OUR PROJECT	6
3	SAMPLE CODING	7
4	ARCHITECTURE DIAGRAM	9
5	DEMO OF PROJECT	11
6	CONCLUSION	15

ABSTRACT

Lumiere Hospital and Clinic Management System is a web-programming project that demonstrates an integrated hospital information experience for public visitors, patients, and clinicians. The application combines a hospital website with an appointment-request workflow, a demonstration Patient Portal, and a demonstration Doctor EMR Console. Public pages present the organisation's philosophy, care pathways, comprehensive check-ups, facilities, speciality centres, medical technology, specialist directory, and booking information. The portal pages present structured clinical-looking views such as demo vital signs, appointment cards, clinical advice, diagnostic-image cards, and an outpatient queue.

The reviewed project is implemented as a Vite multi-page application using HTML5, Tailwind CSS, custom styling, and vanilla JavaScript. Separate entry pages support the public web experience, patient portal, doctor portal, and specialised content pages. The central application script holds a browser-resident HEALTH_PORTAL_STATE object. It serialises this demonstration state under the LUMIERE_HEALTH_PORTAL_STATE localStorage key and updates portal elements through a syncAllPortals routine. This design provides a visible example of DOM updates, hash and pathname routing, form interaction, local persistence, and cross-page user-interface coordination.

The project is a front-end demonstration and must be read within that scope. The inspected code does not provide a production clinical database, authenticated patient accounts, encryption, a FHIR server, live medical-device integration, secure messaging, audit logging, or regulatory controls. All personal names, hospital statistics, vital values, appointments, scans, and clinical advice shown in the pages are demonstration content. The report records the actual project interface, its technical requirements, sample source code, supplied conceptual architecture illustrations, and application demonstrations.

1 SOFTWARE AND HARDWARE REQUIREMENTS

1.1 Software Requirements

Component	Requirement and purpose
Operating system	Windows, Linux, or macOS capable of running Node.js and a current browser.
Runtime and package manager	Node.js with npm for dependency installation and project scripts.
Development server	Vite, configured with multiple HTML entry points for public pages, patient portal, and doctor portal.
Presentation layer	HTML5, Tailwind CSS utility processing, custom CSS, typography, and responsive layouts.
Client-side logic	Vanilla JavaScript in src/app.js for routing, timers, UI updates, forms, and state synchronisation.
Prototype persistence	Browser localStorage under LUMIERE_HEALTH_PORTAL_STATE. It is local to the browser profile and is not a shared database.

1.2 Hardware Requirements

Resource	Minimum working configuration
Processor	Dual-core 64-bit processor or equivalent.
Memory	4 GB RAM minimum; 8 GB RAM recommended for editor, server, and browser use.
Storage	Approximately 1 GB free storage for source files, node modules, build artefacts, and browser cache.
Display	1366 x 768 or higher; 1920 x 1080 is recommended for responsive layout inspection.
Browser	Current Chrome, Edge, Firefox, or Safari with JavaScript and localStorage enabled.

2 FULL VIEW OF OUR PROJECT

Lumiere presents three connected user journeys. The public area works as a hospital and clinic website. It gives visitors access to philosophy, care, check-ups, facilities, centres, specialists, booking, and contact routes. The Patient Portal represents a longitudinal patient view with demo identity data, a vital-sign summary, appointment information, clinical advice, diagnostic items, laboratory information, prescriptions, and tele-consultation navigation. The Doctor EMR Console represents the clinician-facing counterpart with an outpatient queue, encounter timing, patient search, chart actions, and navigation to case-sheet, scan, prescription, laboratory, and consultation views.

2.1 Public Website and Booking Flow

The Vite configuration exposes multiple public HTML pages and provides a consistent navigation structure across the public site. The content is arranged through editorial hero text, cards, tabs, filters, accordions, specialist panels, and a booking form. The check-up catalogue provides category controls and a search field. Facilities and centres present selected speciality information in expandable or tabbed content. The booking page captures a care category, department, preferred date, full name, and phone number before the user submits a request.

2.2 Patient Portal

The Patient Portal is a browser-side demonstration of a health-record experience. It displays a demo patient identity header, status message, vital cards, scheduled consultation, attending clinician advice, ordered investigations, diagnostic-image cards, and patient-navigation tabs. The visible values are generated from the application state and browser storage. This supports the project objective of showing a connected portal interface without implying a live clinical record system.

2.3 Doctor EMR Console

The Doctor EMR Console is organised around outpatient work. Summary cards show consultation and waiting indicators. A timer, patient search field, walk-in action, and schedule table help demonstrate how a clinician-oriented interface could group operational information. Queue rows include demo token number, patient name and identifier, age or sex, scheduled slot, chief complaint, workflow status, and chart-related actions. These elements demonstrate front-end information architecture only.

2.4 Browser State and Routing

The source code defines a HEALTH_PORTAL_STATE object containing active patient data, vitals, prescriptions, SOAP notes, laboratory orders, chat messages, and encounter-timer information. When a page loads, the application restores the state from localStorage, renders relevant elements, attaches user-interface event listeners, starts the encounter timer, and interprets hash or pathname routes. JSON persistence enables later loads in the same browser profile to recover the demonstration state.

3 SAMPLE CODING

The project is built as a browser-delivered multi-page interface. The following representative samples show the way semantic HTML gives a patient-facing card meaning, CSS defines the visual layout, and JavaScript connects visible portal elements to browser-resident demonstration data. The samples are presented for academic understanding of the implementation; all displayed healthcare information remains fictional demonstration content.

3.1 HTML Structure for a Patient Dashboard Card

Listing 3.1 HTML5 patient-summary component

```
<section class="portal-card" aria-labelledby="vitals-heading">
  <div class="card-heading">
    <div>
      <p class="eyebrow">CURRENT VITAL SIGNS</p>
      <h2 id="vitals-heading">Patient health summary</h2>
    </div>
    <span class="status-chip">Physician Verified</span>
  </div>
  <div class="vitals-grid">
    <article class="vital-item">
      <span class="vital-label">Blood Pressure</span>
      <strong data-vital="bp">110/70</strong>
      <small>Optimal range</small>
    </article>
    <article class="vital-item">
      <span class="vital-label">Heart Rate</span>
      <strong data-vital="hr">88 bpm</strong>
      <small>Normal sinus rhythm</small>
    </article>
    <article class="vital-item">
      <span class="vital-label">Blood Oxygen</span>
      <strong data-vital="spo2">100%</strong>
      <small>Room-air measurement</small>
    </article>
  </div>
</section>
```

The HTML sample uses a section landmark, a heading association, grouped article elements, and data attributes. These attributes establish stable query points for JavaScript rendering while the classes provide hooks for presentation rules. The grouping mirrors the card pattern visible in the supplied Patient Portal demonstration.

3.2 CSS Layout and Responsive Rules

Listing 3.2 CSS for portal cards and vital grid

```
.portal-card {
  background: #ffffff;
  border: 1px solid #25211f;
  border-radius: 14px;
  padding: 1.25rem;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

.card-heading {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 1rem;
  border-bottom: 1px solid #25211f;
  padding-bottom: 0.75rem;
}

.vitals-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 0.75rem;
  margin-top: 0.9rem;
}
```

```
.vital-item {
  border: 1px solid #605b56;
  border-radius: 10px;
  padding: 0.85rem;
}

.vital-item strong { display: block; font-size: 1.2rem; }
.vital-label, .vital-item small { display: block; }
@media (max-width: 720px) {
  .vitals-grid { grid-template-columns: 1fr; }
  .card-heading { flex-direction: column; }
}
```

The CSS code defines a reusable boxed card, a flex header, and a grid for the three vital items. The media query changes the grid to one column on narrow displays so that information remains readable. Neutral black, white, and gray values are used in the printed report to keep the style formal and consistent with the required page treatment.

3.3 JavaScript State, Persistence, and Rendering

Listing 3.3 Browser-side state synchronisation routine

```
const STORAGE_KEY = 'LUMIERE_HEALTH_PORTAL_STATE';

function saveStateToStorage() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(HEALTH_PORTAL_STATE));
}

function syncAllPortals() {
  const patient = HEALTH_PORTAL_STATE.activePatient;
  const vitals = HEALTH_PORTAL_STATE.vitals;

  document.querySelectorAll('[data-patient-name]').forEach((element) => {
    element.textContent = patient.name;
  });
  document.querySelectorAll('[data-uhid]').forEach((element) => {
    element.textContent = patient.uhid;
  });
  document.querySelectorAll('[data-vital="bp"]').forEach((element) => {
    element.textContent = vitals.bp;
  });
  document.querySelectorAll('[data-vital="hr"]').forEach((element) => {
    element.textContent = `${vitals.hr} bpm`;
  });
  document.querySelectorAll('[data-vital="spo2"]').forEach((element) => {
    element.textContent = `${vitals.spo2}%`;
  });
  document.querySelectorAll('[data-clinical-advice]').forEach((element) => {
    element.textContent = HEALTH_PORTAL_STATE.soapNotes.assessment;
  });

  saveStateToStorage();
  document.querySelectorAll('.live-sync-time').forEach((element) => {
    element.textContent = new Date().toLocaleTimeString();
  });
}
```

3.4 Source Code Explanation

- The state object supplies the active demonstration identity and values to both portal experiences.
- Data attributes locate output nodes without binding the update routine to a specific page layout.
- JSON storage keeps the current demonstration state in one browser profile; it is not a shared server database.
- The locally formatted timestamp is a presentation cue rather than an authoritative clinical audit event.
- Production healthcare deployment would require secure APIs, authenticated roles, a protected database, validation, and audit controls beyond this browser-only routine.

4 ARCHITECTURE DIAGRAM

The supplied architecture illustrations communicate the intended relationship between the public website, portal experiences, client-side application logic, browser persistence, and runtime environment. The source verification boundary is important: the inspected implementation contains static HTML page entry points, Vite, Tailwind CSS processing, vanilla JavaScript, an in-memory state object, and browser localStorage. Labels such as EHR-FHIR synchronisation, cloud container, encrypted exchange, or live telemetry represent conceptual goals unless a separate secure backend implements them.

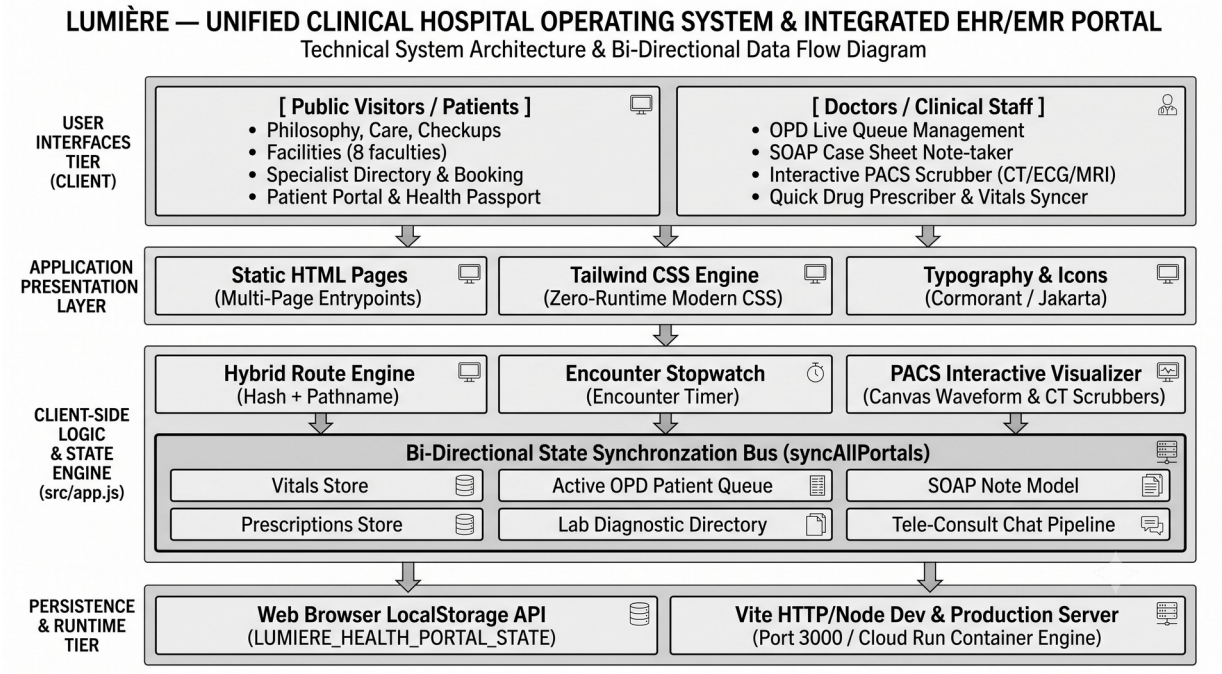


Figure 4.1 Conceptual Lumiere technical system architecture and data flow

The visual groups public or patient interfaces and clinician interfaces above presentation, client-side logic, state synchronisation, local browser persistence, and the Vite runtime. The currently verified behaviour is browser-side state coordination.

4.1 Architecture View

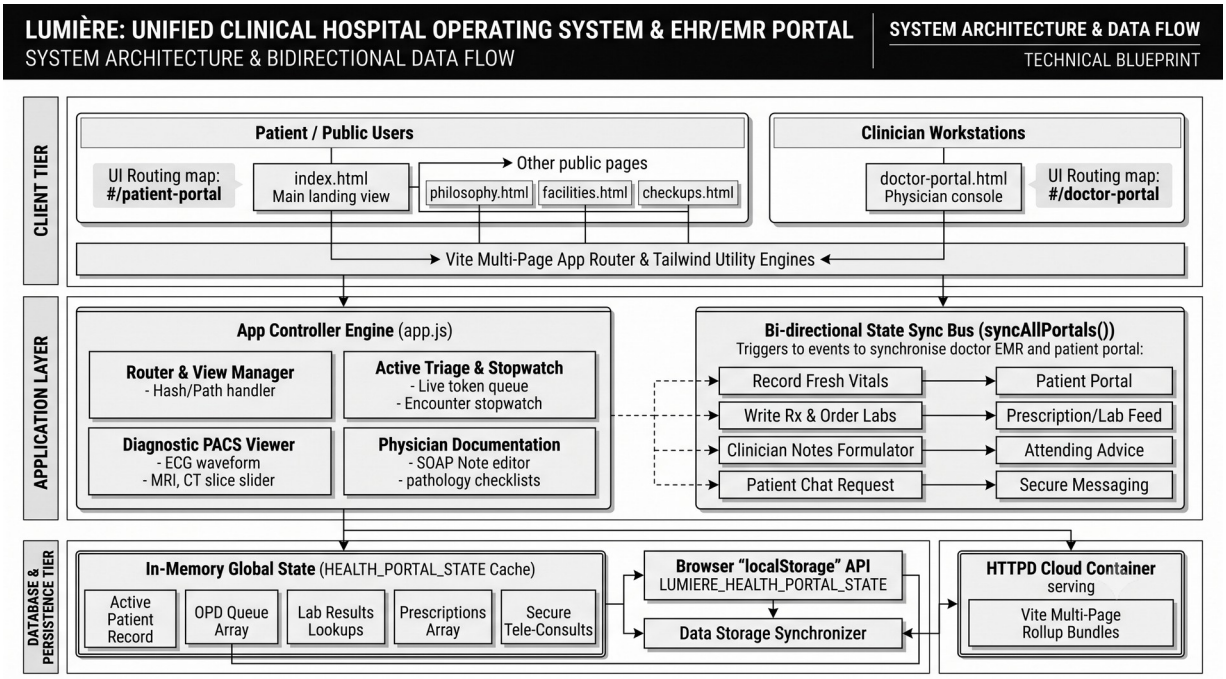


Figure 4.2 Conceptual Lumiere routing, controller, and persistence architecture

The illustration presents an application-controller and sync-bus model. In the available source, routing and portal updates are handled in client-side JavaScript and persisted under a browser localStorage key.

5 DEMO OF PROJECT

The following supplied interface captures document accessible Lumiere pages and meaningful application states. They illustrate the implemented design and browser interface. All content shown in the portal captures is demonstration data and must not be interpreted as a real patient record or verified clinical output.

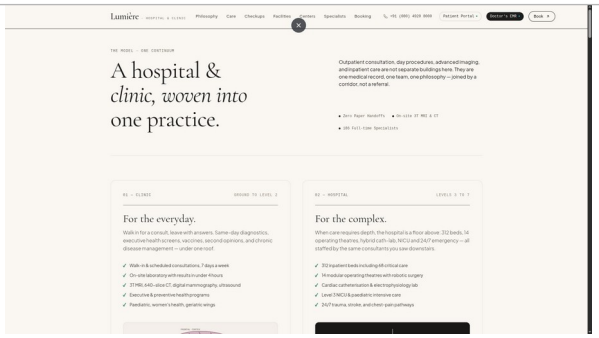


Figure 5.1 Public landing view

The public landing page introduces the clinic and hospital service model.

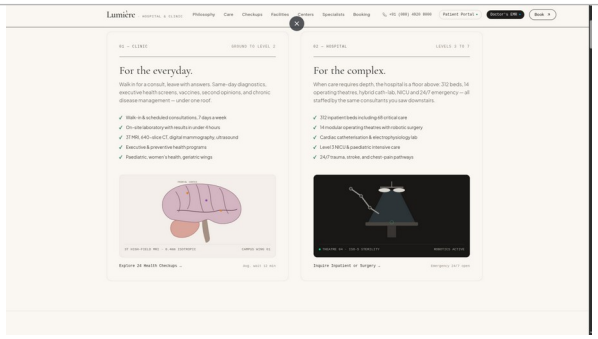


Figure 5.2 Clinic and hospital cards

Paired content cards show facilities, service lists, and paths to further information.

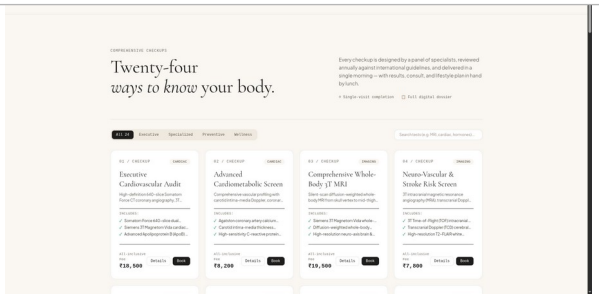


Figure 5.3 Check-up catalogue

Category controls, search, package cards, fees, and booking controls.

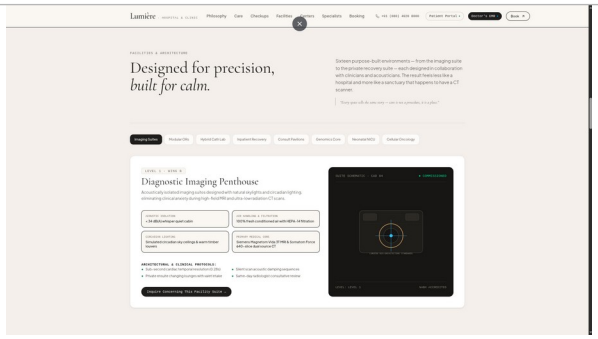


Figure 5.4 Facility detail

A selected diagnostic-imaging facility with tabs, specifications, and inquiry action.

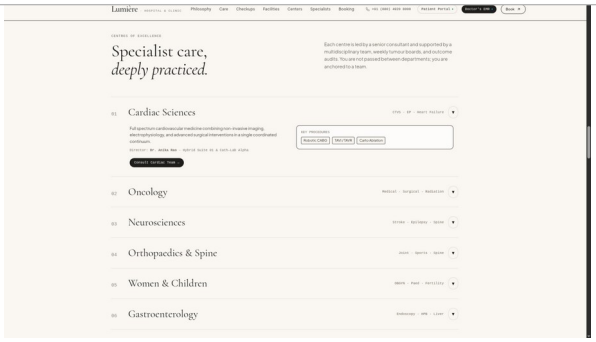


Figure 5.5 Centres of excellence

An expandable speciality directory with an open Cardiac Sciences state.

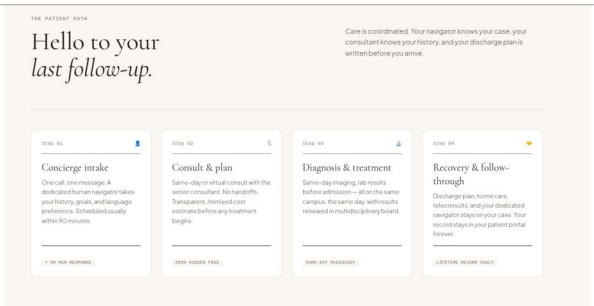


Figure 5.6 Patient path

Four staged cards explain concierge intake, consultation, treatment, and follow-through.

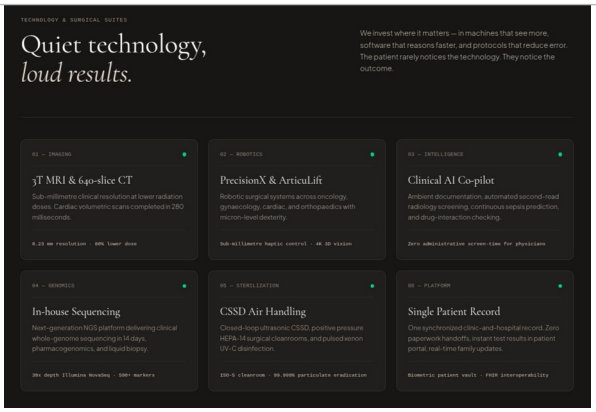


Figure 5.7 Technology suites

A dark section for imaging, robotics, intelligence, genomics, sterilisation, and record concepts.

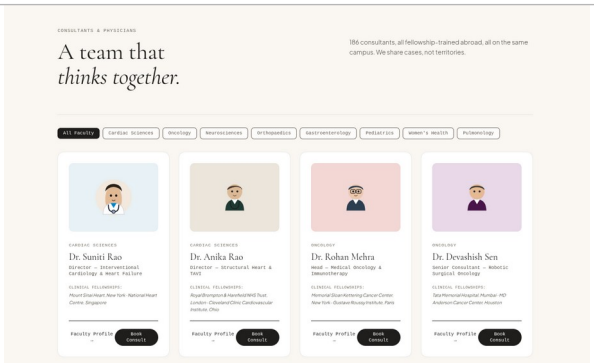


Figure 5.8 Specialist directory

Speciality filter controls and consultant cards with booking actions.

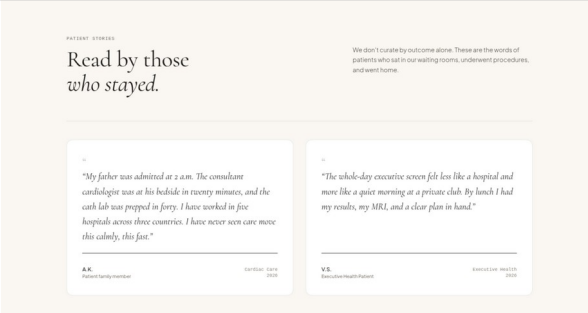


Figure 5.9 Patient stories

Testimonial-style content cards used in the public website presentation.

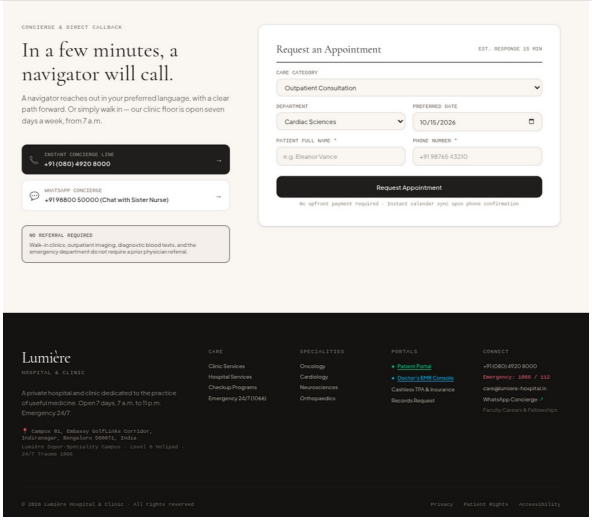


Figure 5.10 Appointment request page

Contact options, the request form, submit action, and navigation footer.

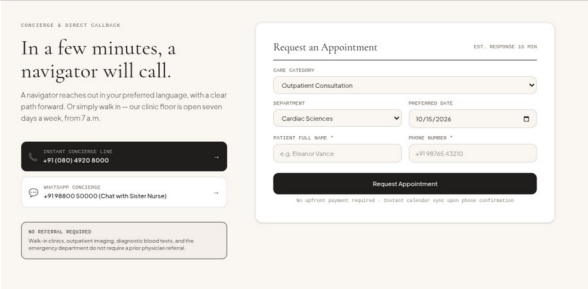


Figure 5.11 Appointment form detail

Care category, department, date, name, phone, and request fields.

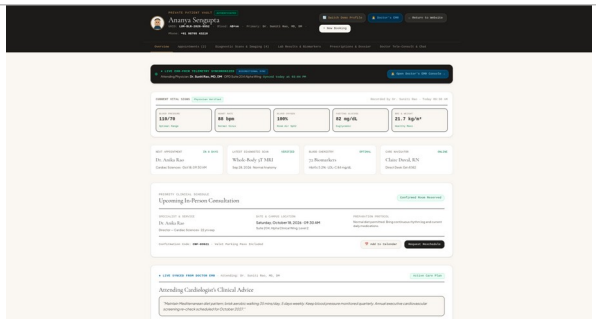


Figure 5.12 Patient Portal overview

Demo identity, vital signs, appointment, status, and clinician-advice sections.

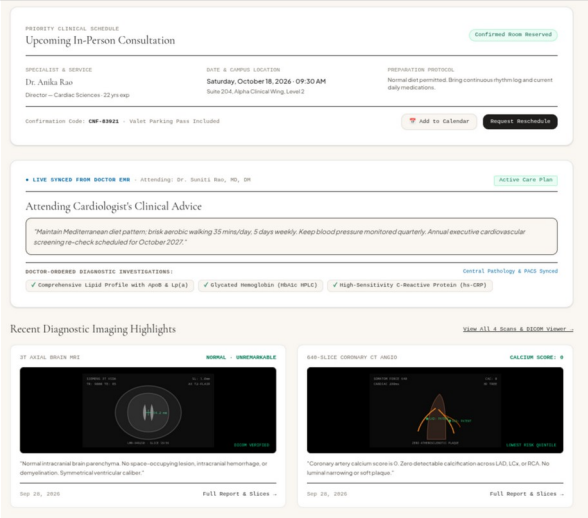


Figure 5.13 Patient advice and diagnostics

Care-plan view, ordered diagnostic chips, and diagnostic-image highlights.

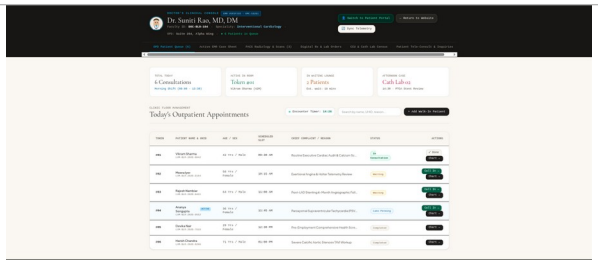


Figure 5.14 Doctor EMR Console

Outpatient summary, timer, patient search, and a workflow queue table.

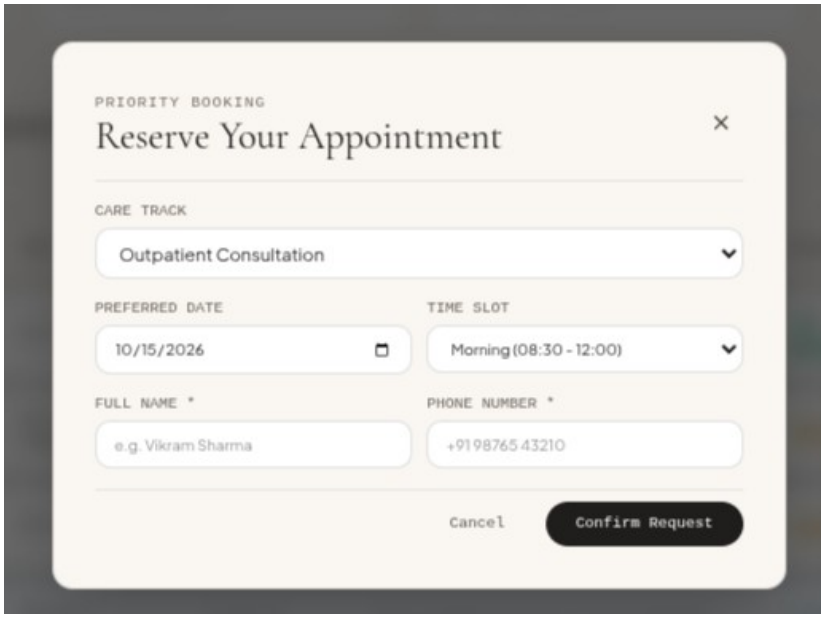


Figure 5.15 Priority appointment modal

The modal demonstrates a compact appointment interaction with care track, preferred date, time slot, contact fields, cancellation, and confirmation controls.

6 CONCLUSION

Lumiere Hospital and Clinic Management System demonstrates a coherent healthcare-oriented web interface that combines public information, appointment requests, a Patient Portal, and a Doctor EMR Console. The project uses Vite multi-page configuration, HTML5, Tailwind CSS, custom styling, and vanilla JavaScript to create a visually consistent experience across diverse page types. The public screens apply cards, filters, tabs, accordions, specialist listings, editorial content, and booking forms. The portal screens demonstrate how clinical-style information can be grouped into readable dashboard and queue layouts.

The application script provides the central technical demonstration. Its HEALTH_PORTAL_STATE object, localStorage persistence, routing logic, timers, and syncAllPortals routine show how data can drive multiple DOM views in a front-end application. The use of structured state and data attributes gives the project a practical example of browser-side coordination between patient and clinician pages.

The reported system must remain positioned as a prototype. It does not currently provide secure server-side storage, real accounts, encrypted communication, standards-based clinical exchange, access control, audit logging, or clinical compliance. Future work should add validated server APIs, authentication with role-based permissions, secure database persistence, transport encryption, input validation, accessibility review, audit trails, and a standards-compliant FHIR interoperability layer before any production healthcare use.

The prototype nevertheless demonstrates several sound web-programming practices. The public pages separate information-rich healthcare presentation from the operational screens used by patients and clinicians. Within the browser, a named state object provides a single source for visible values, while reusable selectors and data attributes permit targeted updates without duplicating the same display logic across every page. This organisation makes the front-end easier to inspect, extend, and explain during a project demonstration.

A production evolution should begin by replacing local browser persistence with a protected server-side data layer. Each request should be authenticated and authorised according to patient, clinician, and administrator responsibilities; clinical values should be validated at the boundary; and every sensitive change should be recorded in an immutable audit trail. Accessibility testing, responsive testing, reliable error states, secure session handling, backup procedures, and standards-based interoperability would then be required to turn the visual prototype into a responsibly engineered healthcare application.

Scope note: No real patient information should be entered into the current prototype. Every clinical-looking value included in the submitted pages serves only as interface demonstration content.